

Iterazione 2

Durante l'iterazione 2 ci siamo concentrati sullo sviluppo dell'UC3, dei relativi test e abbiamo effettuato alcune modifiche riguardanti i nomi di alcune classi.

UC3: Gestione dei costi di prenotazione

Nome caso d'uso	UC3: Gestione dei costi di prenotazione
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: vuole aggiornare i listini prezzi delle strutture in modo flessibile
Pre-condizioni	L'admin è autenticato con i privilegi amministrativi
Garanzie di successo	I costi delle strutture vengono aggiornati e sono immediatamente visibili per le nuove prenotazioni

Scenario principale di successo	1. L'Admin seleziona l'impianto o la specifica struttura da modificare. 2. Il Sistema mostra i parametri attuali. 3. L'Admin seleziona il tipo di tariffazione: "Affitto intera struttura" o "Ingresso singolo" 4. L'Admin imposta il prezzo base (orario o per persona) 5. L'Admin conferma le modifiche. 6. Il Sistema salva le nuove tariffe e le applica alle future prenotazioni. <i>I passi da 1 a 5 vengono ripetuti finché serve.</i> 7. L'admin indica di aver finito.
Estensioni	*a. In qualsiasi momento il sistema fallisce e ha un arresto improvviso. 1. L'admin riavvia il software e ripristina lo stato precedente del sistema. 2. Il sistema ripristina lo stato. 3a. L'Admin inserisce un valore non valido (es. negativo o lettere). 1. Il Sistema mostra un messaggio di errore. 2. L'Admin ripete il passo 3 inserendo i valori corretti

Modello di dominio:

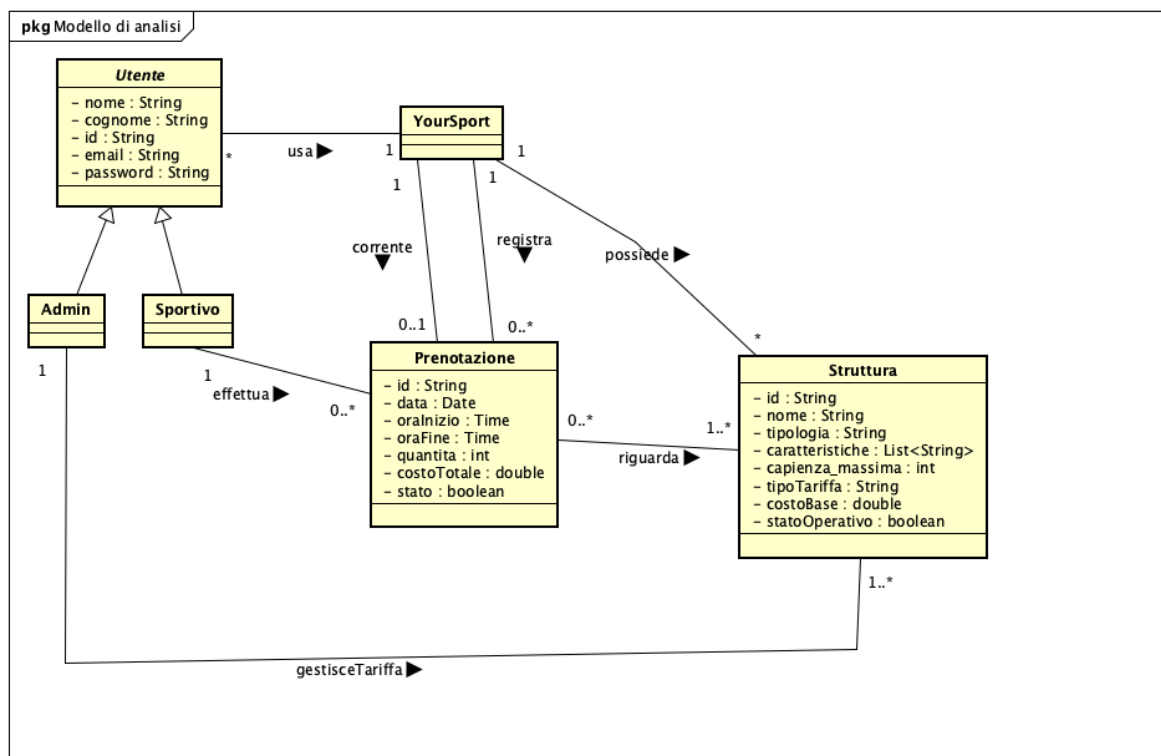
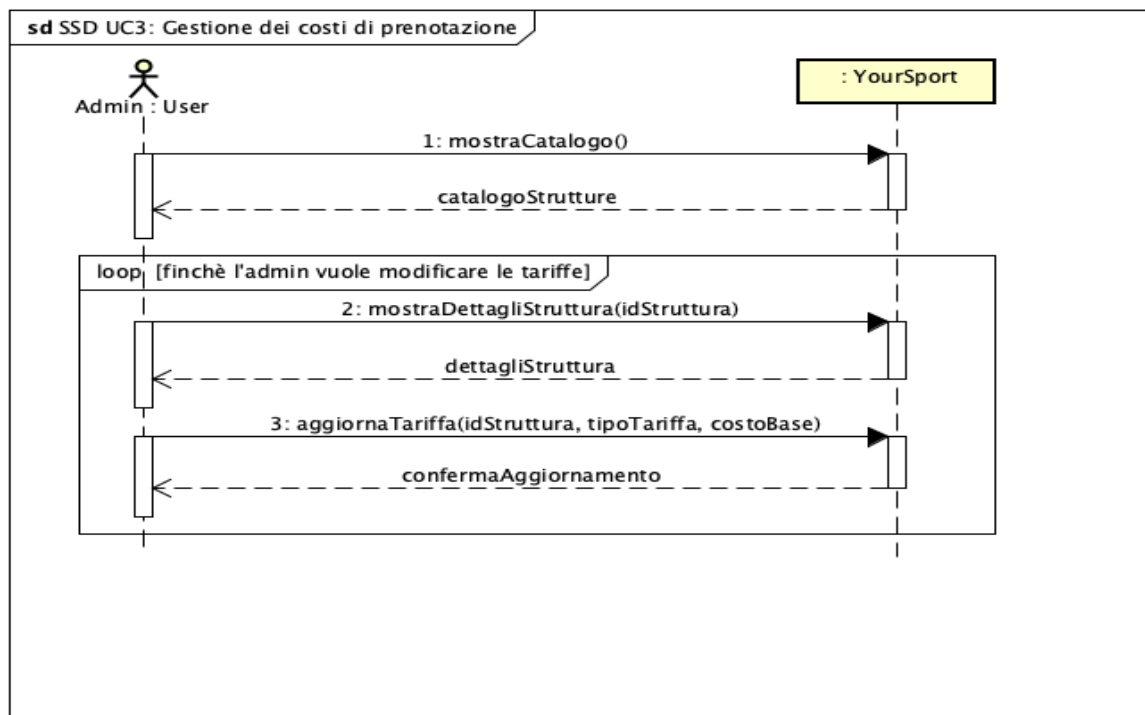


Diagramma di sequenza di sistema:



Contratti delle operazioni:

UC3

Operazioni	mostraCatalogo()
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	L'utente corrente è autenticato nel sistema come Admin
Post-condizioni	Viene restituito l'elenco globale catalogoStrutture contenente tutte le istanze di Struttura

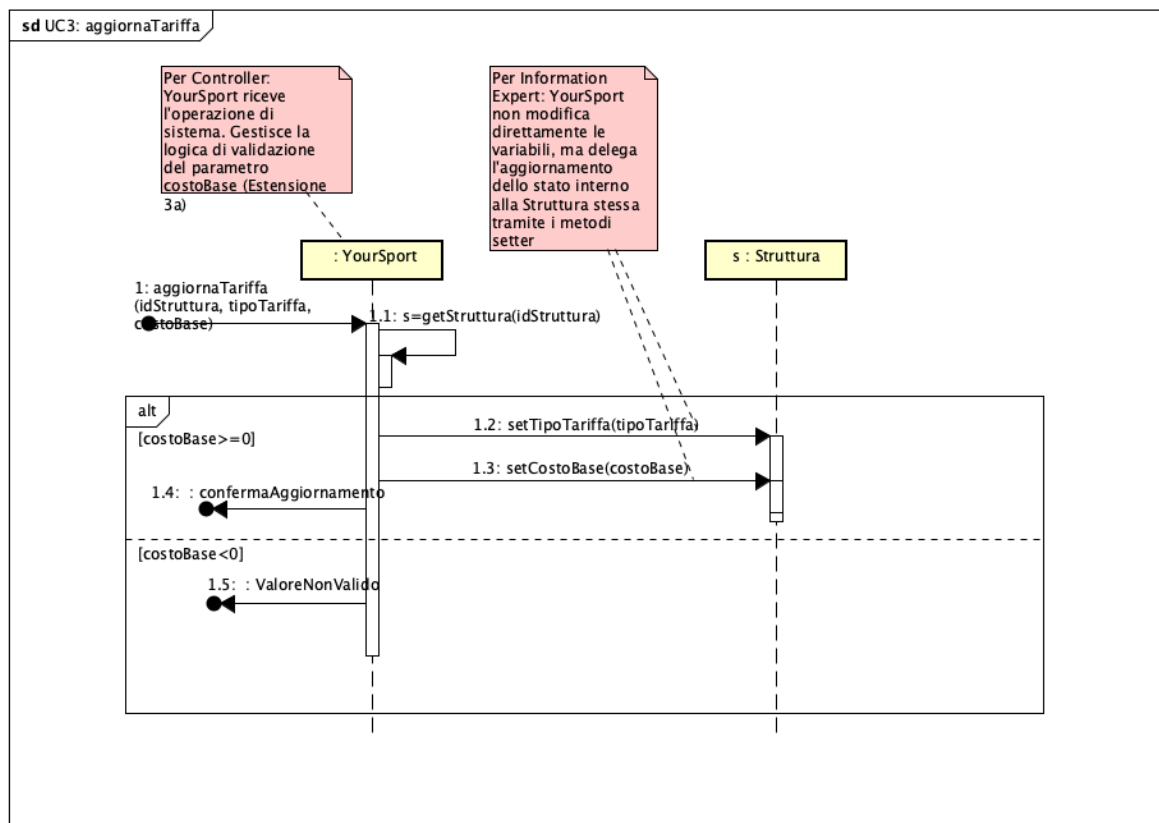
Operazioni	mostraDettagliStruttura(idStruttura: String)
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	Esiste un'istanza s di Struttura il cui identificativo corrisponde a idStruttura
Post-condizioni	Vengono restituiti i dettagli dell'istanza s (inclusi i valori attuali degli attributi tipoTariffa e costoBase)

Operazioni	aggiornaTariffa(idStruttura: String, tipoTariffa: String, costoBase: double)
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	- L'utente corrente è un Admin. - Esiste un'istanza s di Struttura associata a idStruttura. - Il valore di costoBase inserito è un numero valido e positivo

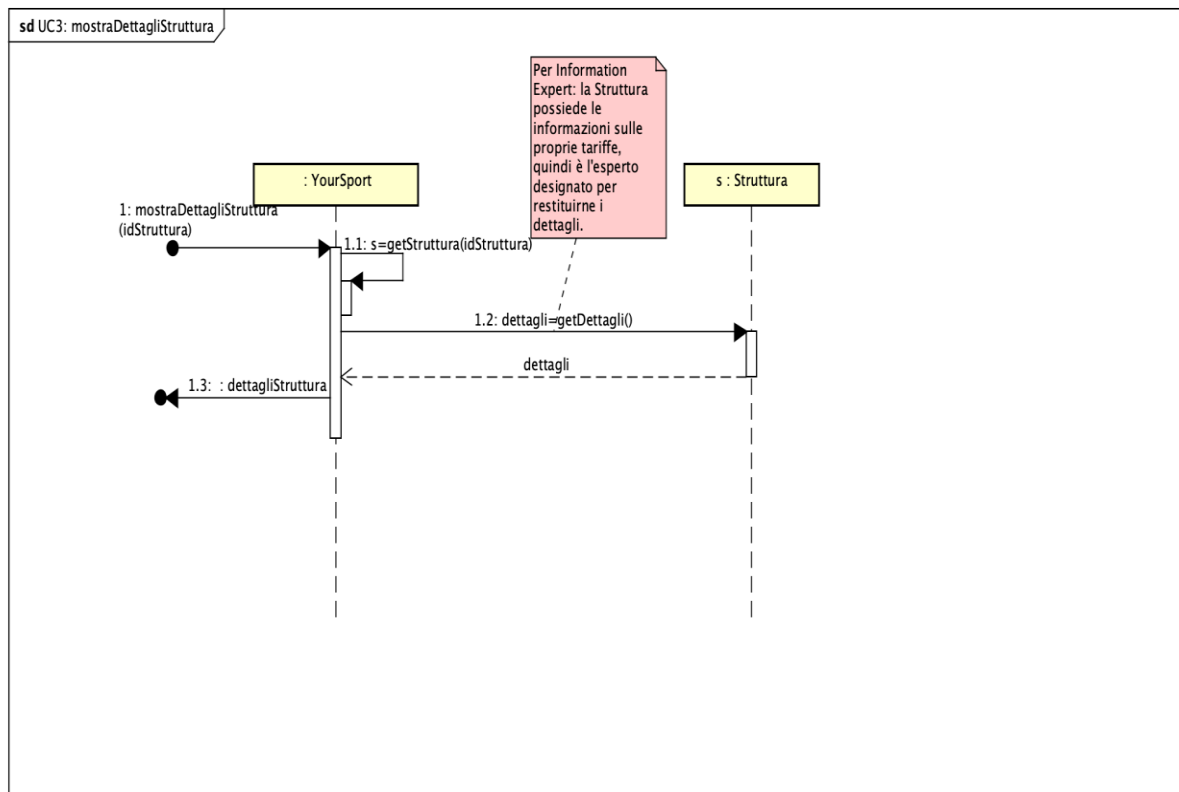
	(gestione estensione 3a del caso d'uso).
Post-condizioni	L'attributo tipoTariffa dell'istanza s è stato modificato e impostato al valore del parametro tipoTariffa.

Diagrammi di sequenza:

-aggiornaTariffa



-mostraDettagliStruttura



Testing:

Test 1: Visualizzazione Catalogo (Mostra Strutture)

Descrizione: Simuliamo l'amministratore che accede al pannello di gestione e richiede di vedere la lista di tutte le strutture presenti nel sistema per decidere quale modificare.

Scopo: Assicurarsi che il sistema (tramite il Controller) restituisca correttamente la collezione delle strutture inizializzate in memoria, verificando che il collegamento al database (in questo caso la lista in memoria) sia attivo.

Realizzazione: Il test chiama il metodo `mostraCatalogo()` e verifica tramite asserzioni (`assertNotNull`, `assertTrue`) che la lista ottenuta non sia vuota o nulla, ma contenga almeno i dati di default (es. il Campo A e la Piscina).

Test 2: Dettagli Struttura (Pattern Information Expert)

Descrizione: Simuliamo l'amministratore che seleziona una struttura specifica (es. "S1", Campo da Tennis) digitandone l'ID per visualizzarne la scheda tecnica e le tariffe attuali.

Scopo: Verificare che il Controller deleghi correttamente la creazione della stringa di riepilogo alla classe Struttura, rispettando il pattern GRASP Information Expert modellato nel Diagramma di Sequenza.

Realizzazione: Il test passa un ID valido al metodo `mostraDettagliStruttura()`. Verifica poi che la stringa restituita contenga i dati corretti (come il nome "Campo A" e il prezzo "20.0") e non messaggi d'errore.

Test 3: Dettagli Struttura Inesistente (Gestione Errore)

Descrizione: Simuliamo l'errore umano in cui l'amministratore digita un ID struttura che non esiste nel sistema (es. "S999").

Scopo: Garantire che il sistema non vada in crash (sollevando una `NullPointerException`) quando cerca una struttura mancante, ma gestisca l'anomalia in modo controllato.

Realizzazione: Il test invoca il metodo passando un ID fittizio e verifica che la stringa restituita corrisponda all'esatto messaggio di errore previsto dal sistema ("Errore: Struttura inesistente").

Test 4: Aggiornamento Tariffa (Flusso Principale/Happy Path)

Descrizione: Simuliamo l'amministratore che decide di cambiare le regole commerciali di una struttura, modificandone con successo sia il tipo di tariffa (es. passandola a PERSONA) sia il costo base (es. alzandolo a 25€).

Scopo: Confermare che l'operazione di sistema aggiornaTariffa modifichi correttamente e permanentemente gli attributi della struttura bersaglio, rispettando le post-condizioni pattuite nei Contratti delle Operazioni.

Realizzazione: Il test legge i dati attuali, chiama il metodo di aggiornamento con nuovi valori validi e infine rilegge la struttura, verificando matematicamente che il prezzo e il tipo siano stati effettivamente sovrascritti.

Test 5: Inserimento Costo Negativo (Estensione 3a)

Descrizione: Simuliamo l'amministratore che digita accidentalmente un prezzo inferiore a zero (es. -5€) durante la fase di aggiornamento della tariffa.

Scopo: Verificare che il sistema applichi rigorosamente il blocco ALT modellato nel Diagramma di Sequenza ([costoBase >= 0]), impedendo di salvare nel sistema prezzi impossibili.

Realizzazione: Il test tenta di impostare un valore negativo. Subito dopo, rilegge l'oggetto Struttura e verifica che il prezzo sia rimasto identico a quello precedente al tentativo, confermando che l'aggiornamento è stato bloccato dal blocco else.

Test 6: Inserimento Tipo Tariffa Non Valido (Programmazione Difensiva)

Descrizione: Simuliamo l'amministratore che inserisce una parola non riconosciuta dal sistema (es. "PISELLO" o "A ORE") al posto dei comandi standard "ORARIO" o "PERSONA".

Scopo: Proteggere l'integrità dell'algoritmo di disponibilità dell'UC2. Se il sistema accettasse stringhe casuali, le future prenotazioni si buggherebbero perché il sistema non saprebbe come calcolare la capienza.

Realizzazione: Il test invia una stringa "spazzatura" come tipo di tariffa. Verifica poi che l'operazione venga scartata istantaneamente e che né il tipo né il prezzo della struttura subiscano alterazioni, confermando la robustezza del codice.

